

Nesne Tabanlı Programlama

Inheritance (Kalıtım) ve Aggregation: IS-A ve HAS-A İlişkileri

Java ile Nesne Tabanlı Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?

Inheritance Temelleri

extends, IS-A ilişkisi, superclass/subclass; neler miras alınır, neler alınmaz; Object class.

Inheritance Türleri

Single, multilevel, hierarchical, multiple (interface ile) ve hybrid inheritance.

Diamond Problem

Java neden class'larda multiple inheritance'a izin vermez; interface'lerle çözüm.

Inheritance Pratikte

Upcasting, downcasting, instanceof, final ile kalıtımı engellemek.

Aggregation (HAS-A)

Bir class'ın başka bir class'ın reference'ını tutması; Employee–Address örneği.

IS-A mı, HAS-A mı?

Inheritance, aggregation ve composition arasında doğru seçim.

Bölüm 1

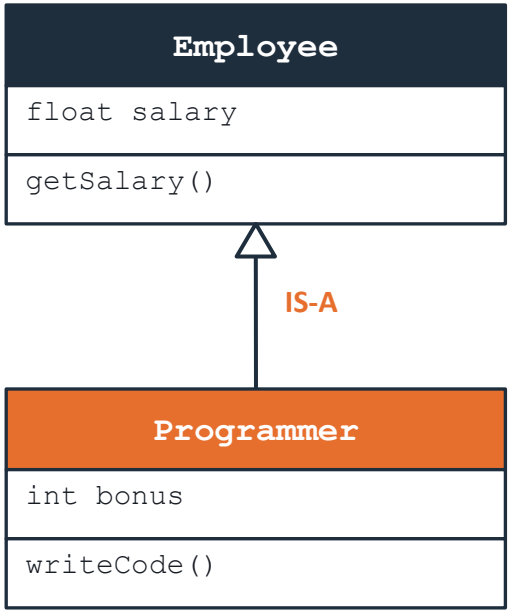


Inheritance Temelleri

Var olan class'ın üzerine yeni class inşa etmek

Inheritance (Kalıtım) Nedir?

- Bir class'ın, başka bir class'ın field ve method'larını **devralması**.
- Devralan: **subclass** (child); devredilen: **superclass** (parent).
- **IS-A** ilişkisini modeller: "Programmer **bir** Employee'dir".
- Java'da `extends` keyword'ü ile yapılır.
- Subclass, miras aldıklarına **yeni** field ve method'lar ekleyebilir ve miras aldığı method'ları **override** edebilir.
- **Neden?** Code reusability, method overriding (runtime polymorphism), genişletilebilir tasarım.



UML: içi boş üçgen ok parent'ı gösterir

Inheritance Terimleri

Terim	Diğer adları	Açıklama
Superclass	Parent class, base class	Özellikleri devredilen class
Subclass	Child class, derived class, extended class	Başka bir class'tan türeyen class
extends	—	Subclass'ı superclass'a bağlayan keyword: class B extends A
IS-A ilişkisi	Parent–child ilişkisi	"B bir A'dır" cümlesi anlamlıysa inheritance uygundur
Reusability	Kodun yeniden kullanımı	Parent'ta yazılmış kod subclass'ta tekrar yazılmadan kullanılır

Syntax: class SubclassName extends SuperclassName { / yeni field ve method'lar */ }*

İlk Inheritance Örneği: Programmer IS-A Employee

```java
class Employee {
    float salary = 40000;
    void work() {
        System.out.println("Çalışıyor...");
    }
}
class Programmer extends Employee {
    int bonus = 10000;                  // yeni field
    void writeCode() {                  // yeni method
        System.out.println("Kod yazıyor...");
    }
}
public class Main {
    public static void main(String[] args) {
        Programmer p = new Programmer();
        System.out.println("Programmer salary is: " + p.salary);
        System.out.println("Bonus of Programmer is: " + p.bonus);
        p.work();                       // Employee'den miras
        p.writeCode();                  // kendi method'u
    }
}
```

Çıktı (Output)

```
Programmer salary is: 40000.0
Bonus of Programmer is: 10000
Çalışıyor...
Kod yazıyor...
```

- **`Programmer`** object'i hem kendi hem de **`Employee`**'nin üyelerine erişir.
- **`salary`** bir **`float`** olduğu için **`40000.0`** yazdırılır.
- **`Employee`** class'ında hiçbir değişiklik yapılmadı → **reusability**.

Neler Miras Alınır, Neler Alınmaz?

Miras alınır

- `public` ve `protected` field ve method'lar
- Aynı package'taysa default (package-private) üyeler
- static üyeler subclass üzerinden erişilebilir (ama override edilmez)

Miras alınmaz / doğrudan erişilemez

- `private` üyeler: object'te **vardır** ama subclass doğrudan erişemez
- **Constructor**'lar miras alınmaz; `super(...)` ile çağrılır
- Instance initializer block'lar

private Üyelere Subclass'tan Erişim

```java
class Account {
    private double balance = 100;  // sadece Account görür
    protected String owner = "Ali";  // subclass görür
    public double getBalance() { return balance; }
}

class Savings extends Account {
    void show() {
        System.out.println(owner);  // OK: protected
        // System.out.println(balance);  // HATA: private
        System.out.println(getBalance());  // public getter ile
    }
}

public class Main {
    public static void main(String[] args) {
        new Savings().show();
    }
}
```

Çıktı (Output)

```
Ali
100.0
```

- **`balance`** Savings object'inde **vardır** ama Savings kodu ona doğrudan dokunamaz.
- Encapsulation korunur: parent kendi verisini kontrol etmeye devam eder.
- Subclass'ların erişmesi gereken üyeler için **`protected`** kullanılır.

Her Class'ın Atası: java.lang.Object

```java
class Student {                    // gizlice: extends Object
    String name;
    Student(String name) { this.name = name; }

    @Override
    public String toString() {         // Object'ten gelen method
        return "Student[" + name + "]";
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Ayşe");
        System.out.println(s);              // toString() çağrılır
        System.out.println(s instanceof Object);
        System.out.println(s.getClass().getSuperclass().getName());
    }
}
```

Çıktı (Output)

```
Student[Ayşe]
true
java.lang.Object
```

- **`extends`** yazılmayan her class otomatik olarak **`Object`**'ten türer.
- Bu yüzden her object'te **`toString()`**, **`equals()`**, **`hashCode()`**, **`getClass()`** vardır.
- **`toString()`** override edilmezse **`Student@1b6d3586`** gibi bir çıktı alınır.

Bölüm 2



Inheritance Türleri

Single, multilevel, hierarchical, multiple ve hybrid

Inheritance Türleri: Genel Bakış

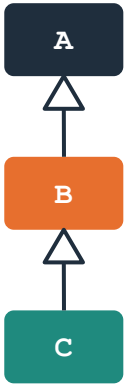
Single



B, A'dan türer

✓ class ile

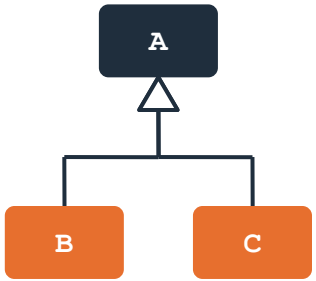
Multilevel



C → B → A zinciri

✓ class ile

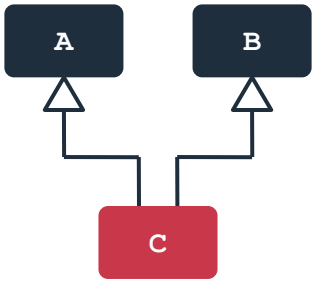
Hierarchical



B ve C, aynı A'dan

✓ class ile

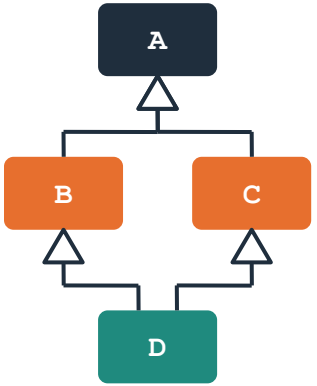
Multiple



C, hem A hem B'den

✗ class ile
✓ interface ile

Hybrid



Birden fazla türün birleşimi

Class ile ancak multiple içermezse

Single Inheritance

```java
class Animal {
    void eat() { System.out.println("eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("barking..."); }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.bark();          // kendi method'u
        d.eat();           // Animal'dan miras
    }
}
```

Çıktı (Output)

```
barking...
eating...
```

- Bir subclass yalnızca **bir** superclass'tan türer.
- En yaygın ve en basit inheritance biçimidir.
- "Dog bir Animal'dır" → IS-A testi geçer.

Multilevel Inheritance

```java
class Animal {
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("barking..."); }
}
class BabyDog extends Dog {
    void weep() { System.out.println("weeping..."); }
}

public class Main {
    public static void main(String[] args) {
        BabyDog d = new BabyDog();
        d.weep();          // BabyDog
        d.bark();          // Dog'dan
        d.eat();           // Animal'dan (dolaylı)
        System.out.println(d instanceof Animal);
    }
}
```

Çıktı (Output)

```
weeping...
barking...
eating...
true
```

- Bir **inheritance zinciri**: BabyDog → Dog → Animal.
- En alttaki class, zincirdeki tüm class'ların üyelerine sahiptir.
- Çok derin zincirler (4-5+ seviye) anlaşılmayı zorlaştırır; sığ tutun.

Hierarchical Inheritance

```java
class Animal {
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("barking..."); }
}
class Cat extends Animal {
    void meow() { System.out.println("meowing..."); }
}

public class Main {
    public static void main(String[] args) {
        Cat c = new Cat();
        c.meow();
        c.eat();
        // c.bark();   -> HATA: Cat, Dog'un method'unu bilmez

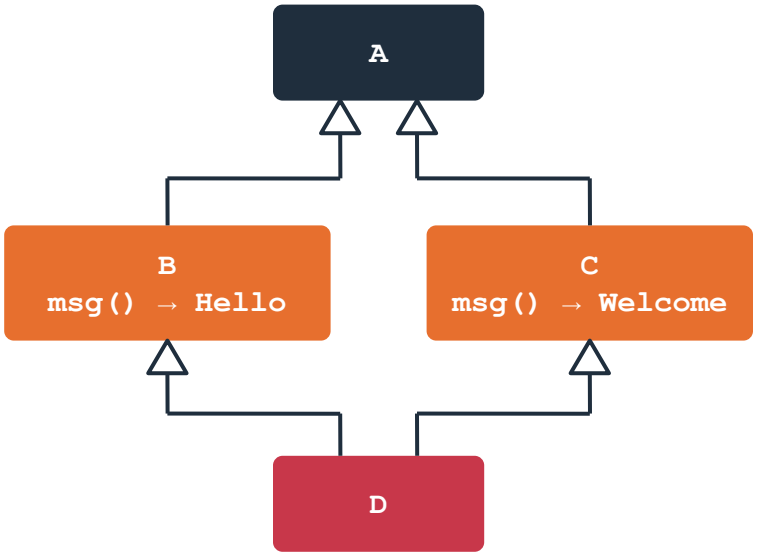
        Animal[] pets = { new Dog(), new Cat() };
        for (Animal a : pets) a.eat();     // ortak davranış
    }
}
```

Çıktı (Output)

```
meowing...
eating...
eating...
eating...
```

- Birden fazla subclass **aynı** superclass'tan türer.
- Kardeş class'lar (Dog, Cat) birbirlerinin üyelerini görmez.
- Ortak parent sayesinde hepsi **`Animal`** tipinde bir array'de tutulabilir.

Diamond Problem



new D().msg() → B'ninki mi, C'ninki mi?

- D hem B'den hem C'den türeseydi ve ikisinde de `msg()` olsaydı, hangisinin çalışacağı **belirsiz** olurdu.
- Şekil elmasa (diamond) benzediği için **diamond problem** denir.
- Java, dili basit tutmak için class'larda multiple inheritance'a **izin vermez**: `class D extends B, C` → compile error.
- Çözüm: bir class **birden fazla interface** implement edebilir.

Multiple Inheritance: Class ile Değil, Interface ile

✗ İki class'ı extend etmek

```
class A {
    void msg() { System.out.println("Hello"); }
}
class B {
    void msg() { System.out.println("Welcome"); }
}
class C extends A, B {      // HATA
}
```

Derleme Hatası

```
error: '{' expected
```

✓ Birden fazla interface'i implement etmek

```
interface Fighter { void attack(); }
interface Weapon  { void use(); }
class Warrior implements Fighter, Weapon {
    public void attack() {
        System.out.println("Kılıçla saldırır");
    }
    public void use() {
        System.out.println("Kılıcı kullanır");
    }
}
Warrior w = new Warrior();
w.attack();
w.use();
```

Çıktı (Output)

```
Kılıçla saldırır
Kılıcı kullanır
```

Interface'lerde Diamond Problem ve Çözümü (Java 8+)

```java
interface A {
    default void hello() { System.out.println("A"); }
}
interface B {
    default void hello() { System.out.println("B"); }
}

class C implements A, B {
    @Override
    public void hello() {          // ZORUNLU: çakışmayı çöz
        A.super.hello();           // A'nınkini çağır
        B.super.hello();           // B'ninkini çağır
        System.out.println("C");
    }
}

public class Main {
    public static void main(String[] args) {
        new C().hello();
    }
}
```

Çıktı (Output)

```
A
B
C
```

- Java 8 ile interface'ler **default method** (gövdeli method) içerebilir.
- İki interface aynı default method'u getirirse compiler class'ı bu method'u **override etmeye zorlar**.
- `InterfaceAdı.super.method()` ile istenen versiyon seçilir: belirsizlik kalmaz.

Hybrid Inheritance

```java
class C { public void disp() { System.out.println("C"); } }
class A extends C { public void disp() { System.out.println("A"); } }
class B extends C { public void disp() { System.out.println("B"); } }
class D extends A { public void disp() { System.out.println("D"); } }

public class Main {
    public static void main(String[] args) {
        C[] objs = { new C(), new A(), new B(), new D() };
        for (C o : objs) {
            o.disp();              // her object kendi disp()'i
        }
        System.out.println(new D() instanceof C);
    }
}
```

Çıktı (Output)

```
C
A
B
D
true
```

- **Hierarchical** (A ve B, C'den) + **multilevel** (D → A → C) bir arada.
- Multiple inheritance içermediği için class'larla yazılabilir.
- Multiple inheritance içeren hybrid yapılar interface'lerle kurulur.

Bölüm 3



Inheritance Pratikte

Gerçek bir hiyerarşi, casting ve final

Gerçekçi Bir Hiyerarşi: Shape, Circle ve Rectangle

Shape.java

```
class Shape {
    String name;
    Shape(String name) { this.name = name; }
    double area() { return 0; }
    void describe() {
        System.out.printf("%s alanı: %.2f%n", name, area());
    }
}
class Circle extends Shape {
    double r;
    Circle(double r) { super("Daire"); this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) {
        super("Dikdörtgen");
        this.w = w; this.h = h;
    }
    @Override double area() { return w * h; }
}
```

main() çıktısı

```
Daire alanı: 3.14
Dikdörtgen alanı: 12.00
Toplam: 15.14
```

- Ortak kod (**`name`**, **`describe()`**) **bir kez** parent'ta yazıldı.
- Her subclass yalnızca **farklı olanı** (alan formülü) override ediyor.
- main: **`Shape[] shapes = { new Circle(1), new Rectangle(3, 4) };`** dizisini gezip **`describe()`** çağırır ve alanları toplar.

Upcasting, Downcasting ve instanceof

```java
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal { void bark() { System.out.println("woof"); } }
class Cat extends Animal { }

Animal a = new Dog();           // upcasting: otomatik
a.eat();
// a.bark();                    // HATA: reference tipi Animal

if (a instanceof Dog) {
    Dog d = (Dog) a;            // downcasting: açık cast
    d.bark();
}
Animal b = new Cat();
if (b instanceof Dog dog) {     // pattern matching (Java 16+)
    dog.bark();
} else {
    System.out.println("b bir Dog değil");
}
// Dog x = (Dog) b;             // ClassCastException!
```

Çıktı (Output)

```
eating
woof
b bir Dog değil
```

- **Upcasting**: subclass → superclass reference; her zaman güvenli, otomatik.
- Reference tipi, **hangi method'ların çağrılabileceğini** belirler.
- **Downcasting**: önce `instanceof` ile kontrol edin; yanlış tip → `ClassCastException`.

final ile Inheritance'ı Sınırlamak

final class: extend edilemez

```
final class Utility {
    static int twice(int x) { return 2 * x; }
}
class MyUtil extends Utility { }    // HATA
```

Derleme Hatası

```
error: cannot inherit from final Utility
```

- JDK'dan örnek: **`String`**, **`Integer`**, **`Math`** final class'lardır.
- Davranışın değiştirilmesi istenmeyen class'lar için.

final method: override edilemez

```
class Account {
    final void audit() { }
}
class Fake extends Account {
    @Override
    void audit() { }                // HATA
}
```

Derleme Hatası

```
error: audit() in Fake cannot override
audit() in Account; overridden method is final
```

- Güvenlik açısından kritik method'lar (doğrulama, denetim) kilitlenebilir.

Inheritance Ne Zaman Kullanılmalı?



Kullanın

- "B **bir** A'dır" cümlesi her zaman doğruysa (IS-A)
- Subclass, parent'ın **yerine** kullanılabiliyorsa (parent'ın beklendiği her yerde sorunsuz çalışıyorsa)
- Ortak davranış gerçekten paylaşılıyorsa



Kaçının

- Sadece birkaç method'u tekrar kullanmak için `extends` yazmak
- "Car extends Engine" gibi IS-A olmayan ilişkiler
- Çok derin hiyerarşiler
- Bu durumlarda **composition / aggregation** (HAS-A) tercih edin

Bölüm 4

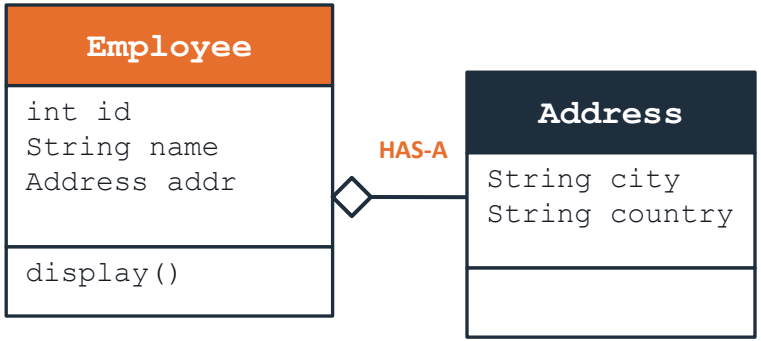


Aggregation (HAS-A)

Bir class'ın başka bir class'ı içermesi

Aggregation Nedir?

- Bir class'ın, başka bir class'ın object'ine **reference** (entity reference) tutmasıdır.
- **HAS-A** ilişkisini modeller: "Employee'nin **bir** Address'i **vardır**".
- İçerilen object (Address) **bağımsız** yaşayabilir; başka object'lerle **paylaşılabilir**.
- Inheritance olmadan kodu yeniden kullanmayı sağlar: iş, içerilen object'e **delege** edilir.
- Association'ın zayıf bağlı özel bir hâlidir.



UML: içi boş elmas (◇) aggregation'ı, içi dolu elmas (◆) composition'ı gösterir.

Aggregation ile Kodun Yeniden Kullanımı

```java
class Operation {
    int square(int n) { return n * n; }
}

class Circle {
    Operation op;  // HAS-A
    Circle(Operation op) { this.op = op; }
    double area(int radius) {
        return Math.PI * op.square(radius);  // delegation
    }
}

public class Main {
    public static void main(String[] args) {
        Operation op = new Operation();
        Circle c1 = new Circle(op);  // aynı Operation
        Circle c2 = new Circle(op);  // paylaşılıyor
        System.out.println(c1.area(5));
        System.out.println(c2.area(1));
    }
}
```

Çıktı (Output)

```
78.53981633974483
3.141592653589793
```

- **`Circle`**, kare alma işini kendisi yazmak yerine **`Operation`**'a **delege** eder.
- **`Operation`** dışarıda oluşturulup verildiği için iki Circle tarafından **paylaşılır**.
- π için **`3.14`** yerine **`Math.PI`** kullanmak daha doğrudur.

Gerçek Dünya Örneği: Employee HAS-A Address

```
Address.java / Emp.java
class Address {
    String city, country;
    Address(String city, String country) {
        this.city = city;
        this.country = country;
    }
}

class Emp {
    int id;
    String name;
    Address addr;                          // Emp HAS-A Address
    Emp(int id, String name, Address addr) {
        this.id = id;
        this.name = name;
        this.addr = addr;
    }
    void display() {
        System.out.println(id + " " + name);
        System.out.println("  " + addr.city + ", " + addr.country);
    }
}
```

main() çıktısı

```
111 Varun
  Gümüşhane, Türkiye
112 Arun
  Gümüşhane, Türkiye
112 Arun
  Torul, Türkiye
```

- main: iki çalışan **aynı `Address`** object'ini alıyor; sonra **`gms.city = "Torul"`** yapılıyor.
- Paylaşılan object değişince **her iki** çalışan da yeni adresi görür.
- Çalışan silinse de adres yaşamaya devam eder → aggregation.

IS-A mı, HAS-A mı? Doğru İlişkiyi Seçmek

✗ Car extends Engine (araba bir motor mu?)

```
class Engine {
    void start() {
        System.out.println("Motor çalıştı");
    }
}
class Car extends Engine { }

Car car = new Car();
car.start();
```

Çıktı (Output)

```
Motor çalıştı
```

- Derlenir ve çalışır ama **modelleme yanlış**.
- Car, motorun tüm public method'larını da dışarı açar.

✓ Car HAS-A Engine

```
class Engine {
    void start() {
        System.out.println("Motor çalıştı");
    }
}
class Car {
    private Engine engine = new Engine();
    void start() { engine.start(); }
}
new Car().start();
```

Çıktı (Output)

```
Motor çalıştı
```

- Car, yalnızca istediği davranışı dışarıya sunar.
- Motor kolayca değiştirilebilir (ör. ElectricEngine).

Inheritance, Aggregation ve Composition

Kriter	Inheritance	Aggregation	Composition
İlişki	IS-A	HAS-A (zayıf)	HAS-A (güçlü)
Java'da	extends / implements	Dışarıdan verilen reference	İçeride oluşturulan reference
Bağımlılık (coupling)	En yüksek	Düşük	Orta
Parçanın ömrü	—	Bütünden bağımsız	Bütüne bağlı
Paylaşım	—	Parça paylaşılabilir	Parça paylaşılmaz
Örnek	Dog IS-A Animal	Employee HAS-A Address	Car HAS-A Engine
UML gösterimi	Boş üçgenli ok	Boş elmas ◇	Dolu elmas ◆

Alıştırma: Çıktı Nedir?

1

Soru 1

```
class P { int x = 1; }
class Q extends P { int y = 2; }
Q q = new Q();
System.out.println(q.x + q.y);
```

2

Soru 2

```
class P {
  void hi() { System.out.print("P"); }
}
class Q extends P {
  void hi() { System.out.print("Q"); }
}
P p = new Q();
p.hi();
```

3

Soru 3

```
class P { }
class Q extends P { }
P p = new P();
System.out.println(p instanceof Q);
System.out.println(new Q() instanceof P);
```

4

Soru 4

```
class Box { int v; }
class H { Box b; H(Box b) { this.b = b; } }
Box box = new Box();
H h1 = new H(box), h2 = new H(box);
h1.b.v = 5;
System.out.println(h2.b.v);
```

Özet

Inheritance (IS-A)
extends ile tek bir class'tan türetme; private üyeler ve constructor'lar doğrudan miras alınmaz; her class Object'ten türer.

Türler
Single, multilevel, hierarchical class'larla; multiple (ve onu içeren hybrid) yalnızca interface'lerle.

Pratik
Upcasting otomatik, downcasting instanceof ile güvenli; final class/method kalıtımı sınırlar.

Aggregation (HAS-A)
Başka bir object'e reference tutmak; IS-A cümlesi doğru değilse inheritance yerine HAS-A tercih edin.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).